PASSWORD STRENGTH CHECKER

1.ABSTRACT

Password security plays a vital role in protecting personal and organizational information in the digital world. With the increasing use of online services such as banking, e-commerce, social media, and educational portals, the need for secure authentication mechanisms has become more important than ever. Passwords are the first line of defense against unauthorized access. However, many users create weak passwords that can be easily guessed or cracked using cyberattack techniques such as brute-force attacks, dictionary attacks, and phishing attacks.

The Password Strength Checker is a Python-based application designed to analyze and evaluate the security level of passwords. The system examines a password using several security criteria, including minimum length, uppercase letters, lowercase letters, numbers, and special characters. Based on these checks, a score is calculated and the password is classified into different strength levels such as Very Weak, Weak, Medium, Strong, and Very Strong.

The project helps users understand the importance of creating secure passwords and provides recommendations for improving password quality. By using Regular Expressions (Regex) and Python programming concepts, the application efficiently validates passwords and generates meaningful feedback. This project serves as an educational tool for understanding cybersecurity concepts and secure password practices.

2.INTRODUCTION

In today's digital era, almost every online service requires user authentication. Passwords are the most commonly used method for verifying user identity. From social networking websites and online banking systems to educational portals and

business applications, passwords help ensure that only authorized users can access sensitive information.

Despite their importance, many users continue to create weak passwords because they are easier to remember. Common examples include names, dates of birth, phone numbers, and simple sequences such as "123456" or "password". Such passwords can be cracked within seconds using modern hacking techniques.

Cybercriminals use various methods to obtain passwords, including:

- Brute-force attacks
- Dictionary attacks
- Credential stuffing
- Social engineering
- Phishing attacks

A strong password significantly reduces the risk of unauthorized access. Security experts recommend passwords that contain a combination of uppercase letters, lowercase letters, numbers, and special characters. Longer passwords are generally more secure because they increase the number of possible combinations.

The Password Strength Checker project is developed to evaluate password quality and provide instant feedback to users. The application helps users understand whether their password is secure enough and suggests improvements when necessary. The project also demonstrates practical applications of Python programming and Regular Expressions in cybersecurity.

3.OBJECTIVES

The primary objective of the Password Strength Checker is to evaluate password security and encourage the use of strong passwords.

Specific Objectives

1. To analyze passwords using predefined security rules.
2. To determine whether a password meets minimum security requirements.

3. To calculate a security score based on password characteristics.
4. To classify passwords into different strength categories.
5. To provide feedback and suggestions for improvement.
6. To improve user awareness regarding password security.
7. To demonstrate the practical use of Python programming.
8. To utilize Regular Expressions for pattern matching.
9. To reduce the risk of password-related security breaches.
10. To create a simple and user-friendly password validation system.

4.TECHNOLOGY USED

Python Programming Language

Python is a high-level programming language known for its simplicity, readability, and versatility. It is widely used in cybersecurity, web development, machine learning, and data analysis.

Features of Python

- Easy to learn and use
- Simple syntax
- Large standard library
- Cross-platform compatibility
- Open-source software
- Strong community support

Regular Expressions (Regex)

Regular Expressions are patterns used to search and match text strings. In this project, Regex is used to validate password components

Requirement	**Regex Pattern**

Uppercase Letter	[A-Z]
Lowercase Letter	[a-z]
Numeric Digit	[0-9]
Special Character	[!@#$%^&*()]

Benefits of Regex

- Fast validation process
- Accurate pattern matching
- Reduced code complexity
- Easy implementation

Development Tools

Visual Studio Code

A lightweight code editor used for writing and executing Python programs.

PyCharm

An advanced Integrated Development Environment (IDE) designed specifically for Python development.

IDLE

Python's default development environment for beginners.

Operating Systems Supported

- Windows
- Linux
- macOS

5.METHODOLOGY

The Password Strength Checker follows a systematic approach for evaluating password security.

Step 1: Password Input

The user enters a password through the keyboard.

Step 2: Password Length Verification

The system checks whether the password contains at least eight characters.

Importance

Longer passwords are generally harder to crack because they increase the number of possible combinations.

Step 3: Uppercase Letter Check

The system verifies whether the password contains at least one uppercase letter.

Example

Valid:

- Password123

Invalid:

- password123

Step 4: Lowercase Letter Check

The system verifies the presence of lowercase letters.

Example

Valid:

- PASSWORDabc

Invalid:

- PASSWORD123

Step 5: Numeric Digit Check

The system checks whether the password contains numerical digits.

Example

Valid:

- Password123

Invalid:

- Password

Step 6: Special Character Check

The system verifies the presence of special symbols.

Example

Valid:

- Password@123

Invalid:

- Password123

Step 7: Score Calculation

Each satisfied condition contributes one point to the overall score.

Condition	**Score**
Length ≥ 8	1
Uppercase Letter	1
Lowercase Letter	1
Number	1
Special Character	1

Maximum Score = 5

Step 8: Strength Classification

The system determines password strength according to the final score.

Score	Strength
0–1	Very Weak
2	Weak
3	Medium
4	Strong
5	Very Strong

Step 9: Display Results

The password strength and suggestions are displayed to the user.

6.ALGORITHM

Step 1: Start the program.

Step 2: Read the password entered by the user.

Step 3: Initialize a variable score = 0.

Step 4: Check the length of the password.

- If the password length is greater than or equal to 8 characters, increment score by 1.
- Otherwise, add a suggestion to increase the password length.

Step 5: Check for at least one uppercase letter (A–Z).

- If present, increment score by 1.
- Otherwise, suggest adding uppercase letters.

Step 6: Check for at least one lowercase letter (a–z).

- If present, increment score by 1.
- Otherwise, suggest adding lowercase letters.

Step 7: Check for at least one numeric digit (0–9).

- If present, increment score by 1.
- Otherwise, suggest adding numbers.

Step 8: Check for at least one special character.

- If present, increment score by 1.
- Otherwise, suggest adding special characters.

Step 9: Calculate the final score.

Step 10: Determine the password strength based on the score:

- Score 0–1 → Very Weak
- Score 2 → Weak
- Score 3 → Medium
- Score 4 → Strong
- Score 5 → Very Strong

Step 11: Display the password strength level.

Step 12: Display improvement suggestions if any security condition is not satisfied.

Step 13: Stop the program.

7.Program

```
 # Password Strength Checker
import re
def check_password_strength(password):
    strength_points = 0
    suggestions = []

    # Rule 1: Minimum length
```

```
if len(password) >= 8:
    strength_points += 1
else:
    suggestions.append("Use at least 8 characters.")

# Rule 2: Uppercase letter
if re.search(r"[A-Z]", password):
    strength_points += 1
else:
    suggestions.append("Add at least one uppercase letter.")

# Rule 3: Lowercase letter
if re.search(r"[a-z]", password):
    strength_points += 1
else:
    suggestions.append("Add at least one lowercase letter.")

# Rule 4: Number
if re.search(r"[0-9]", password):
    strength_points += 1
else:
    suggestions.append("Add at least one number.")

# Rule 5: Special character
if re.search(r"[!@#$%^&*(),.?\":{}|<>]", password):
    strength_points += 1
```

```
    else:
        suggestions.append("Add at least one special character.")

    # Strength evaluation
    if strength_points == 5:
        strength = "Very Strong"
    elif strength_points == 4:
        strength = "Strong"
    elif strength_points == 3:
        strength = "Medium"
    elif strength_points == 2:
        strength = "Weak"
    else:
        strength = "Very Weak"

    return strength, suggestions

# User input
password = input("Enter your password: ")

# Check strength
strength, suggestions = check_password_strength(password)

# Display result
print("\nPassword Strength:", strength)
```

```
# Suggestions
if suggestions:
    print("\nSuggestions to improve password:")
    for item in suggestions:
        print("-", item)
else:
    print("Your password is secure.")
```

8.Output

Example 1

Enter your password: Password@123

Password Strength: Very Strong

Your password is secure.

Example 2

Enter your password: 12345

Password Strength: Very Weak

Suggestions to improve password:

- Use at least 8 characters.

- Add at least one uppercase letter.

- Add at least one lowercase letter.

- Add at least one special character.

Example 3

Enter your password: Password123

Password Strength: Strong

Suggestions to improve password:

- Add at least one special character.

Example 4

Enter your password: password

Password Strength: Weak

Suggestions to improve password:

- Add at least one uppercase letter.

- Add at least one number.

- Add at least one special character.

Example 5

Enter your password: PASSWORD123

Password Strength: Medium

Suggestions to improve password:

- Add at least one lowercase letter.

- Add at least one special character.

CONCLUSION

The Password Strength Checker is a simple yet effective cybersecurity application that promotes secure password practices. The project highlights the importance of strong passwords in protecting digital information from unauthorized access and cyber threats.

By evaluating password characteristics such as length, uppercase letters, lowercase letters, numbers, and special characters, the system accurately determines password strength and provides meaningful suggestions for improvement. The implementation of Regular Expressions enables efficient validation and pattern matching.

The project demonstrates practical applications of Python programming in cybersecurity and serves as an educational tool for students and beginners. Future enhancements may include password breach detection, entropy analysis, password generation, and graphical user interfaces. Overall, the Password Strength Checker contributes to improved cybersecurity awareness and encourages users to adopt stronger password habits.